Sanket: Swecha Fashions readiness audit

Audit date: 25 October 2026, India. **Meeting:** 26 October 2026. **Verdict: suitable for a transparent, supervised demonstration; not ready for an unqualified production or accuracy promise.**

Sanket is a working application with real forecasting, stock ingestion, scenario calculations, and decision records. It is not working perfectly. Several defects can materially mislead a buyer: repeated imports double sales, forecast dates fail to align with actuals, INR revenue is displayed 83 times too high, and parts of the interface claim models and signal confidence that the running services did not substantiate.

The best proposition for Swecha tomorrow is a measured proof of value using their own data, after the critical defects below are fixed. Avoid describing the sample benchmark as Swecha's accuracy, promising guaranteed savings, or demonstrating unattended purchasing. This audit confirmed 12 issues and retained 205 HTTP observations; these are observations, not 205 independent passing tests.

1. What was actually run

- Current local repository, real FastAPI backend, PostgreSQL with pgvector, Redis, an arq worker, Vite frontend, and the actual Amazon Chronos T5 Small model on CPU.
- Fresh audit database on port 55432 and Redis on 56379; backend on 8010 because an unrelated program already occupied 8000; ML on 8001; frontend on 5173.
- Migrations through revision 0021 succeeded. Application and reconciliation queries used `sanket_app`, with `rolsuper=false` and `rolbypassrls=false`. Elevated access was used only to initialize/migrate the dedicated database.
- Two separate fashion workspaces: `swecha-audit` deliberately tested repeated uploads; `swecha-audit-clean-run` imported training sales once. Test subscriptions were explicitly provisioned as fixtures after the normal unpaid workspace returned 402. This does not validate payment processing.
- The in-app browser failed to navigate twice, so Chrome was used for the visible walkthrough. Screenshots are from this run, not previous audits. Initial screenshots intentionally show the repeated-upload stress workspace.
- No production source files or existing business records were changed. Local synthetic recommendations and alerts were accepted/dismissed/resolved as test records. No supplier order, payment, email, or WhatsApp message was sent.

Swecha context

Swecha's public pages list kurtis, gowns, leggings and palazzos, and Hyderabad-area stores including Boduppal and Alwal. The straight-cut kurti page includes sizes through 4XL and 5XL. These informed the sample categories, locations, and the size-validation check; they are not evidence of Swecha's sales volumes or inventory.

Sources: Swecha public business and catalogue, straight-cut kurti and sizes, store locations. The direct homepage fetch initially timed out; search-indexed pages were available. Confirm their current POS/ERP and store structure in the meeting.

2. Dataset and evaluation method

All audit sales and stock are deterministic synthetic fixtures, not Swecha customer data or a public retail benchmark. Random seed: 25102026. Prices and costs are test assumptions in INR.

Four products, six SKU/size series, and two inventory locations were created. Five series had 104 training weeks; one had 12 weeks. Patterns covered stable demand, growth, annual seasonality, intermittent demand, a four-week zero-sales interruption, and short history. Training dates: 2 September 2024 to 24 August 2026. Eight subsequent weeks, 31 August to 19 October 2026, were withheld for all six SKUs: 48 test points.

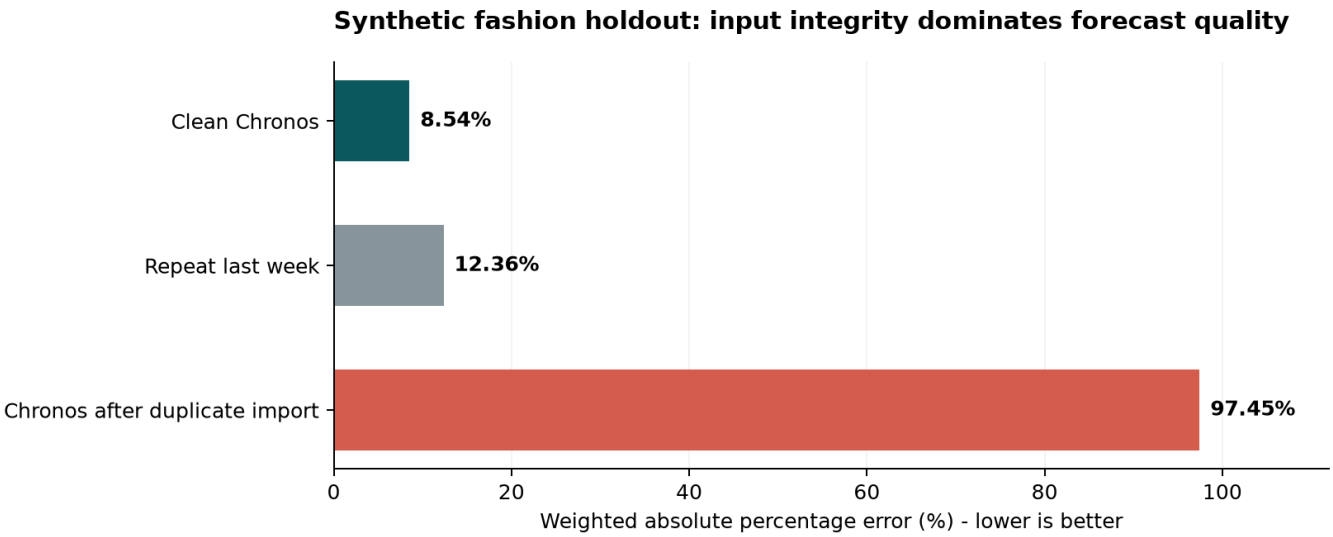
The file contained 532 training rows. The importer accepted 459 and rejected 73 explicit zero-quantity rows. The loader subsequently filled internal missing weeks with zeros; boundary zero weeks can disappear. Clean imported sales totalled 17,482 units; the repeated-upload workspace contained 918 rows and 34,964 units. Inventory totalled 717 units across 12 SKU/location positions.

Predictions and benchmark metrics were frozen before the held-out sales were imported. WAPE is `sum(abs(actual - p50)) / sum(actual) * 100`; lower is better. Last-value, trailing-four-week mean, and 52-week seasonal baselines were calculated independently. A single historical cutoff is a diagnostic, not a production validation study.

Calendar caveat: the model returned Sundays while sales are Monday-labelled weekly buckets. To measure the numerical forecast separately from this defect, the eight forecast steps were paired with the eight intended withheld weeks in chronological order. These are horizon-aligned diagnostic scores, not a successful end-to-end date-matched accuracy result. The built-in accuracy endpoint still returned no matched rows after held-out actuals were uploaded.

3. Is it predicting well enough?

Clean sample result	Measured value
Forecast engine actually used	Chronos zero-shot, no trained tenant artifact
SKUs / withheld points	6 / 48
Portfolio WAPE	8.54%
Repeat-last-week baseline WAPE	12.36%
P10-P90 interval coverage	81.25% (39 of 48 points)
Repeated-upload portfolio WAPE	97.45%
First measured warm 8-week API forecast	1.98 seconds



SKU	Pattern	Chronos WAPE	Last-value WAPE	Interpretation
KURTI-M	Stable	8.08%	7.32%	Simpler baseline wins
KURTI-XL	Growing	3.80%	4.36%	Beats last value; four-week mean was 3.61%

SKU	Pattern	Chronos WAPE	Last-value WAPE	Interpretation
GOWN-M	Seasonal	13.80%	41.46%	Beats last value; seasonal baseline was better at 10.57%
GOWN-XL	Intermittent	100.00%	100.00%	Median predicted zero; actual total was 19 units
LEGGING-M	Zero-sales interruption	8.03%	14.29%	Recovers level; seasonal baseline was 5.90%
PALAZZO-XL	12 weeks of history	11.25%	11.11%	Short-history baseline slightly better

This is encouraging for a pilot on regular sellers. It does not show that the engine always beats a simple method: it beat last value on three SKUs, lost on two, and tied on one. Portfolio averages hide the intermittent failure. Interval coverage near 80% across only 48 points is not proof of calibration; stable, stockout and short-history cohorts each covered only 62.5% of their points.

Do not relabel 8.54% WAPE as a guaranteed 91.46% accuracy. No real client sales, rolling-origin study, promotion calendar, returns history, or actual stockout availability data were supplied. The zero-sales interruption was detected heuristically, not from verified stock availability. No festival-specific learning or external-signal benefit was demonstrated.

4. Confirmed defects and their causes

F01 - Critical: repeated sales imports double demand

Re-uploading exactly the same CSV, with the same order IDs, imported all 459 valid rows again. Database reconciliation showed two copies of every accepted order. The importer creates a new upload ID and unconditionally adds `HistoricalSale` rows; it does not deduplicate sales by source/order/line identity. See `backend/app/services/integrations/file_import.py:696` and the sales branch near line 772.

The hypothesis was that duplicate rows inflated model input. Independent tenant-scoped sums confirmed exactly twice the units, and the same sample's WAPE rose from 8.54% to 97.45%. Data Quality correctly detected 459 duplicates and called them critical, but forecasting remained available. Its repair preview also failed with HTTP 500 (F12); prevention and a forecast gate are still needed.

Recommended verification: import a known file twice and prove row counts, units, revenue and forecasts remain unchanged; preserve genuinely different sales lines. Do not use the repeated-upload workspace as a client demo.

F02 - High: forecast dates and actuals do not align

The loader correctly anchors weekly history to Mondays, but zero-shot future dates use the unanchored pandas `W` offset, which anchors to Sunday. A training cutoff of Monday 24 August produced the first forecast date Sunday 30 August, while the intended next sales bucket starts Monday 31 August. The accuracy router joins Monday-truncated actuals to forecast dates by exact date string.

Observed result: actuals were uploaded after predictions were saved, yet `/forecast/accuracy` still returned `rows: []`. Relevant code: `backend/ml/sanket_ml/data/loader.py:204`, `backend/ml/sanket_ml/inference/zero_shot.py:211`, and `backend/app/routers/forecast_accuracy.py:139` onward.

Recommended verification: one weekly calendar convention from ingestion to inference, storage, exports and scoring; a forecast made before the cutoff must join the correct subsequent week, including the final week's transactions.

F03 - High: INR sales revenue is multiplied by 83

All fixture SKU prices are INR. The sales API returned year gross revenue 13,091,162 in the stress workspace; the INR screen displayed 1,086,566,446, exactly 83 times larger. `SalesAnalytics.tsx` calls `formatPrice` without a source currency; `useFormattedCurrency.ts:6` and `:33` default to USD, and the currency store defaults to an exchange rate of 83.

This is separate from the doubled-import defect. Even perfectly deduplicated INR sales are displayed incorrectly. Recommended verification: reconcile raw sales totals with screen, charts, tables and exports for INR and USD; carry currency explicitly and define how mixed-currency aggregation works. Do not demonstrate the current revenue totals as Swecha economics.

F04 - High: forecast screen makes unsupported live claims

The service returned `source=chronos_zero_shot`; the screen said `Live ensemble - TFT, DeepAR, Prophet+`. A static insight claimed Q3 sell-through was 12% above the prior season, and displayed a positive trend even though there were no live trend samples. The labels come from hardcoded model and insight maps in `frontend/src/pages/Forecasts.tsx:62` and `:101`, not the response.

The page also takes only the first four grouped SKUs (`:588`), reports four SKUs forecasted, and omits two charts despite a six-series response. Forecast export included all 48 eight-week rows. Recommended verification: render actual source, horizon, signal status and all series (with pagination if needed); no comparative percentage without its underlying calculation.

F05 - High: no signals is displayed as high signal confidence

The live signal insert failed with `no partition of relation trend_signals found for row`. Only May-August 2026 monthly partitions were created by the baseline SQL; October's partition was absent. Existing rolling partition maintenance covers other tables, not this one.

The Trend Analysis screen then showed zero signals, a stable demand interpretation, and `High (100%)` agreement. Its calculation uses `(1 - volatility) * 100` with a zero-volatility empty default (`frontend/src/pages/TrendAnalysis.tsx:374-384`). Missing evidence is being interpreted as perfect agreement. The hybrid result itself was more honest: zero samples and zero trend adjustment.

Recommended verification: provision current/future trend partitions through the migration/maintenance path; test source failure and empty data; display unavailable/insufficient evidence instead of 100% agreement. Live weather/social/search lift was not validated in this audit.

F06 - High for Swecha: real catalogue sizes are rejected

Creating a 5XL fashion SKU returned 422, explicitly listing only XS, S, M, L, XL, XXL and XXXL. The same enum also excludes 4XL (`backend/app/services/industry_context.py:64`). Swecha publicly lists both sizes. Make the size system configurable or extend/normalize it before importing the full catalogue. Preserve size variants instead of silently flattening them.

F07 - Medium: zero-sales guidance contradicts import validation

Data Quality recommends explicit zero rows to distinguish no sales from missing data. The sales importer rejects quantity zero. In this test 73 such rows were skipped, including the four-week interruption. Internal gaps were later filled by the loader, but this does not preserve boundary zeros or the distinction between missing and observed zero demand.

Recommended verification: an agreed weekly aggregate format that accepts observed zero demand, separately from transaction validation, and retains availability/stockout information.

F08 - Medium: unsold products do not receive forecasts

An additional, valid size-L SKU was created without sales. The next successful response still contained only the original six series and omitted that SKU. The zero-shot path requires at least eight observations; it is zero-shot with respect to model training, not zero-history demand inference. The marketing promise that a never-sold SKU receives a usable curve is not supported by this path. Explicitly report omitted SKUs and define a validated analogue/new-product method before promising cold-start coverage.

F09 - Medium: stock health can look safe before decision refresh

Inventory reconciled correctly to 717 units, but the initial screen showed zero critical positions, `Cover pending` and `No action`, while hybrid forecasting had generated shortage alerts. Without an open decision, the display uses an absolute threshold of 20 units rather than actual demand coverage (`frontend/src/pages/Inventory.tsx`, `classifyCoverage`). Refreshing decisions in the clean workspace produced 11 actionable records. Tie stock health directly to demand/lead time, or clearly mark risk as unknown while context is pending.

For Swecha's multiple stores, note that this tested forecast aggregates demand by SKU. Location-level stock is stored, but the stock screen reuses the SKU-level decision context across locations; the final clean view showed the same 26-day cover for both palazzo locations despite different stock. Store-level demand/coverage and transfer allocation require separate validation before being promised.

F10 - Medium: estimated ROI is labelled realized immediately on approval

Before approvals, ROI was zero and forecast improvement was null, which is appropriate. Accepting one synthetic recommendation immediately produced `total_value_realized=144258.44`, including model-based protected revenue and an avoided-stockout count. No supplier order, receipt or observed avoided stockout occurred. The screen nevertheless says the figures are not modelled or projected. The arithmetic is an estimate against the frozen recommendation baseline, not verified business savings. Label estimated/committed versus observed realized value separately.

F11 - Medium: dashboard readiness overstates WhatsApp delivery

The dashboard hardcodes `Digest enabled` in `frontend/src/components/dashboard/PlanningBriefDashboard.tsx:379`. The actual notifications API reported `enabled=false`, `provider=console`, `configured=false`; the WhatsApp page correctly said delivery was not connected. Use the actual configuration on the dashboard. Preview works; delivery remains untested and unconfigured.

F12 - High: duplicate auto-fix preview fails

The non-destructive request `POST /data-quality/fix` with `action=dedupe_sales` and `confirm=false` returned 500. The actual backend error was `UndefinedFunctionError: function min(uuid) does not exist`. The repair query aggregates `min(historical_sales.id)` even though the identifier is a UUID (`backend/app/services/data_quality_audit.py`, `apply_fix`). PostgreSQL 16 in this stack does not provide that aggregate. No deletion was attempted. Replace the UUID aggregate with a deterministic supported survivor selection and verify preview count, intended survivor, tenant scope, and the actual repair on disposable fixtures before relying on this button.

5. Feature-by-feature outcome

Area	Evidence and outcome
Signup, sign-in, industry context	Working locally. Unpaid access returned 402; a test subscription was then granted explicitly.
Tenant and industry isolation	Anonymous products 401; foreign product 404; wrong-industry header and pharma page 403. RLS-scoped query saw zero foreign-tenant products. Limited evidence, not a full security audit.
Onboarding	Industry selection reflected fashion; skip setup worked; readiness API worked. Full onboarding completion was not walked through.
Products and SKUs	Product/SKU creation and size curve worked; 5XL rejected; unsold SKU omitted from forecasts.
CSV sales	Mapping/analyze and import worked with row errors reported; repeated import duplicated sales. Invalid row returned 200 with zero imported and an explicit row error.
Excel inventory	Analyze and import accepted all 12 rows; stock remained 717 across 12 positions.
Forecast and confidence	48 probabilistic rows for six SKUs, ordered finite quantiles; conservative reasons for intermittent and short history. Calendar and display defects remain.
Hybrid/background worker	202 submission followed by confirmed completed status through Redis/arq; real ML baseline, shortage scan, persisted accuracy snapshot. No trend adjustment because no signals.
Inventory and EOQ	Levels reconciled. EOQ endpoint succeeded for all six valid SKU queries; initial missing-parameter 422 was corrected in the test, not treated as a product failure.
Decisions	Refresh generated 11 records. Accept and dismiss succeeded; repeated accept correctly returned 409. No purchase order sent externally.
Scenarios	Baseline/no-change, +30% demand, four-week supplier outage and -20% price scenarios completed. UI demand preset updated the comparison. Effects are assumption-based simulations, not causal proof.
Alerts	Hybrid generated shortage alerts; acknowledgement and resolution succeeded.
Sales analytics and financial impact	APIs returned data; INR sales display conversion is wrong. Financial impact is modelled exposure, not observed loss.
Forecast accuracy	Screen shows readiness, not invented measured accuracy. Endpoint failed to match uploaded actuals because of the weekly date mismatch.
Trends and external signals	APIs reachable but empty; ingestion failed on missing partition; zero evidence displayed as high confidence.
Data quality	Correctly detected duplicates and missing weeks. Repair preview failed 500 due to `min(uuid)`; destructive repair application was not exercised.
Exports	Product/SKU/stock/signal/alert CSV endpoints succeeded. Forecast XLSX opened successfully: one sheet, seven columns, 48 data rows plus header.
ROI	Read and approval-linked arithmetic worked. Realized-value wording is stronger than the evidence.
Integrations	Catalogue shows 42 sources, six labelled live. CSV/Excel verified. Shopify/WooCommerce/SQL source connections were not exercised with client systems; Tally/POS/ERP catalogue entries are not all implemented.
WhatsApp, billing, webhooks	Configuration/preview/read paths checked. No real delivery, payment, subscription purchase or external webhook delivery was attempted.

6. Verification and evidence limits

Verification logs, machine-readable responses, fixtures, independent metrics and screenshots are retained beside this report. Exact outcomes are recorded in `verification-summary.json`.

- Frontend typecheck: exit 0.
- Frontend lint: exit 0, with 27 existing warnings and zero errors.
- Frontend unit tests: 17 files / 92 tests passed.
- Production frontend build: exit 0; bundle-size warnings remain.
- ML tests: 34 passed, one deprecation warning.
- Backend `uv run pytest`: **371 passed, 1 failed, 164 warnings**, exit 1, in 1648.11 seconds. The failure was `tests/test_external_signals.py::test_regional_demand_connector_fetch`: `assert len(samples) > 5`, actual `0`. An isolated rerun passed in 7.83 seconds. The cause of the full-suite-versus-isolated difference remains unproven; the suite is not green. A separate entitlement diagnostic passed all 18 tests.
- Authenticated WebSocket handshake, tenant identity and ping/pong passed after setting the documented `VITE_WS_BASE_URL` for the separate API port. Sales-event fan-out was not tested. The earlier Sales Analytics screenshot shows the initial transport configuration's offline indicator.
- All 192 rows examined across four captured forecast responses had finite, nonnegative, ordered P10/P50/P90 values. This checks numerical validity, not forecast skill.
- Repository Playwright E2E suite was not run unchanged: it targets port 5173 with seeded dev credentials and an API proxy fixed to the already-occupied port 8000. The current browser walkthrough and direct API checks supplied live functional evidence instead. This is not equivalent to full E2E coverage.

Performance measured only small, warm, local requests. No concurrent load, multi-day soak, production deployment, backup/restore, full accessibility compliance, mobile browser matrix, or client connector authentication was established. Screenshots show desktop presentation only. Exact business forecasts, stockout reduction and financial return require Swecha data and prospective validation.

7. Tomorrow's meeting: what to show and say

Lead with Swecha's planning problem: the right style and size available in the right store, with fewer excess units. Show a labelled sample stock file, the reconciled 717-unit inventory view, how a demand or supplier scenario changes a plan, the data-quality warnings, and the reasoning behind a recommendation. The prototype can make the workflow tangible without promising an outcome already achieved.

Suggested wording: **"Sanket can bring your sales and stock into one planning view, generate SKU-level demand ranges, and help a buyer compare replenishment choices. I propose a limited pilot on your own data, where we measure it against your current method before relying on it for orders."**

If asked about accuracy: **"In a small synthetic fashion test, numerical eight-step forecasts had 8.54% weighted absolute error versus 12.36% for repeating last week. That is a diagnostic, not your expected accuracy, and the date-matching defect must be corrected before using the built-in score."**

Avoid demonstrating the current INR revenue totals as correct, describing the hardcoded model banner as real, claiming live social trend lift, promising 4XL/5XL support today, or claiming an unsold style can be forecast without an analogue/history. Do not show the deliberately duplicated workspace as a clean demonstration.

Proposed pilot acceptance criteria (recommendations, not agreed client commitments)

1. Fix repeat-import safety, weekly date alignment, currency handling, and unsupported live claims before using recommendations operationally.
2. Obtain 12-24 months of dated SKU/size/store sales where available, returns, promotions, stock availability, current stock, inbound POs, supplier lead times, costs and MOQ. Confirm current POS/ERP and include 4XL/5XL in the canonical catalogue.
3. Start with one or two stores and a small representative SKU cohort, including intermittent and new styles. Preserve a true holdout and use several rolling cutoffs across relevant seasons.
4. Agree a forecast horizon matching lead time plus review period. Compare against last value, trailing mean, seasonal baseline and the client's current method on identical cutoffs; choose any model/baseline routing using training data only.
5. A possible go/no-go target is at least 10% relative portfolio WAPE reduction over an agreed baseline, with no material degradation of important size/SKU cohorts, and acceptable uncertainty coverage. This is a proposed test gate, not a guarantee or a universal industry standard.
6. Run four weeks in shadow mode with human approval and reconcile recommendations to actual orders, receipts, fill rate and inventory outcomes before calling savings realized. Do not rely on the current ROI label as proof.

8. Demo access and reproducibility

Frontend: `http://127.0.0.1:5173/login`. Clean workspace: `swecha-audit-clean-run`; local audit email: `fashion.audit-clean-run@example.com`; local-only password: `AuditFashion2026!`. These are disposable audit credentials, not client access. The clean workspace now includes the held-out observations, one intentionally unsold SKU, and the explicitly labelled test action records.

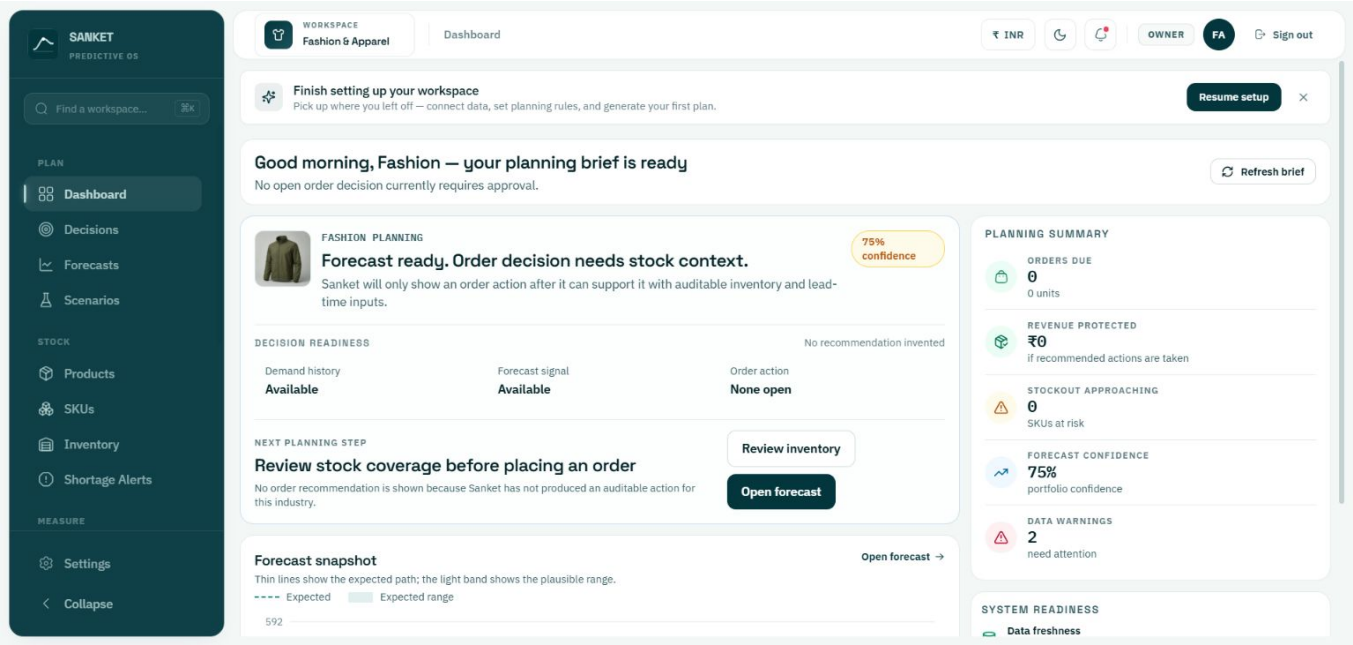
If the services stop, run `start-audit-demo.ps1` in this folder. It starts only the dedicated containers and local service processes and sets both HTTP and WebSocket URLs. Allow up to 90 seconds for model startup. It uses the existing audit database, dependencies and cached model; it is not a deployment script. Check `/health`, ML `/health`, and a forecast before the meeting. Original `.env` files remain unchanged.

Reproduction files: `run_audit.py`, `provision_fixture.py`, `deep_checks.py`, `action_checks.py`, `api-evidence.json`, `database-reconciliation.json`, `holdout-metrics.json`, and the corresponding `clean-run/` files. The audit runners create records and deliberately exercise duplicate ingestion; they are not idempotent demo reset scripts. Run them only against a fresh dedicated audit environment.

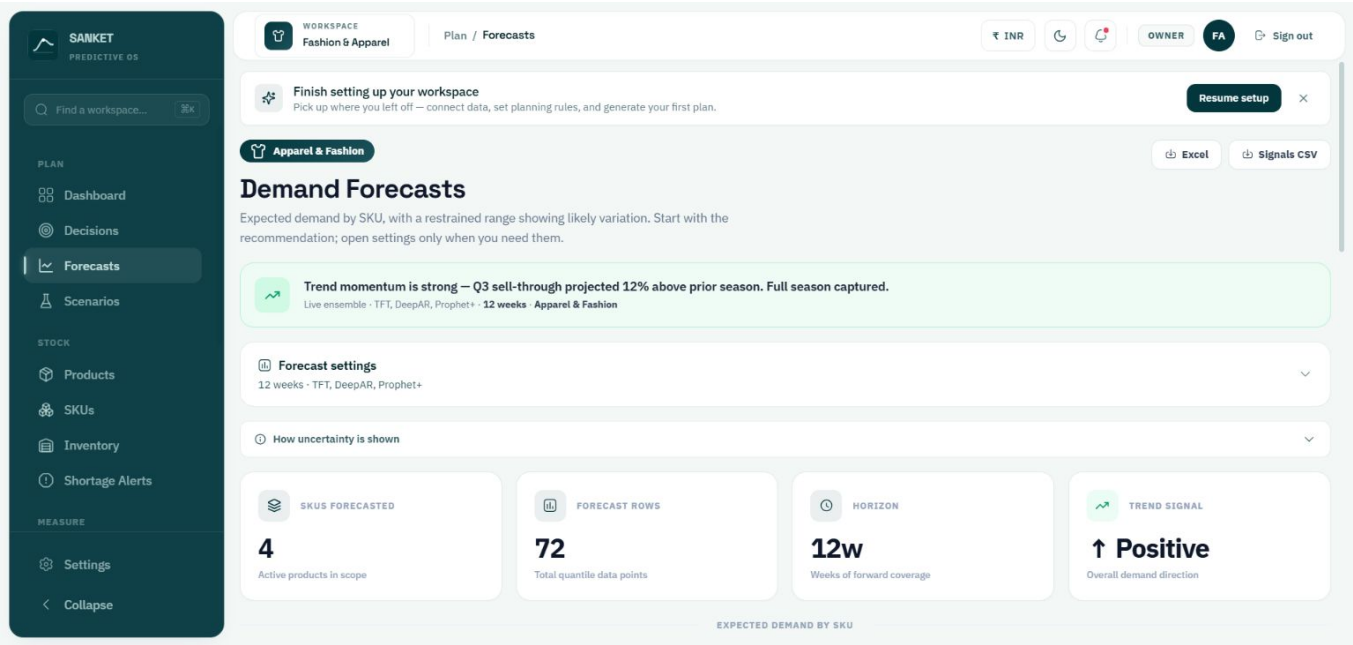
9. Screenshot evidence

Captured in the repeated-upload stress workspace. Values are synthetic, including the deliberately inflated inputs. These screenshots prove the displayed states; they do not represent Swecha business performance.

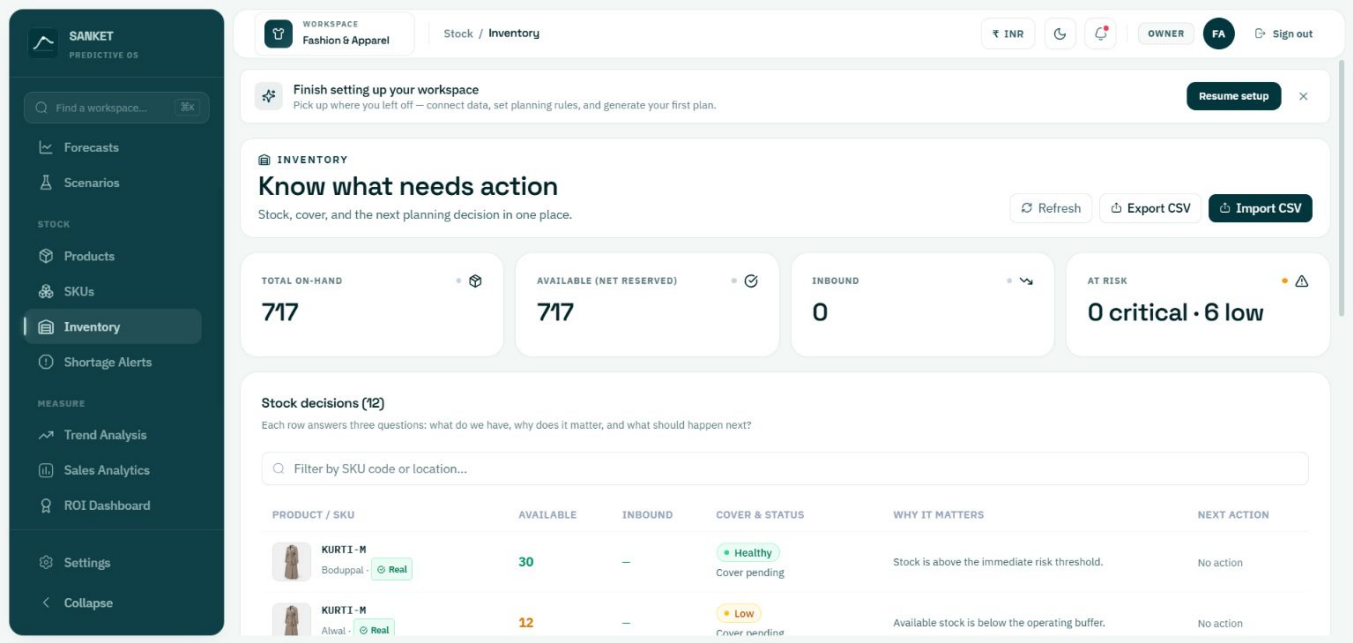
1. Dashboard renders; readiness overstates WhatsApp delivery.



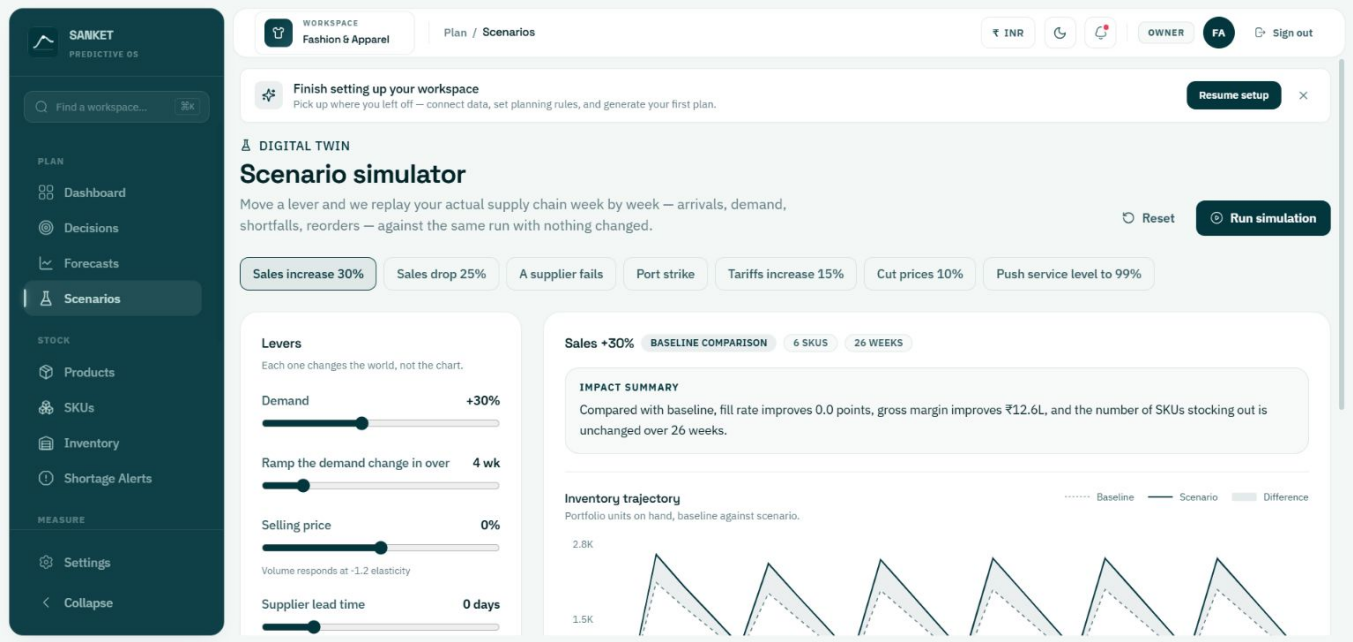
2. Forecast page: unsupported model/uplift copy and only four displayed SKUs.



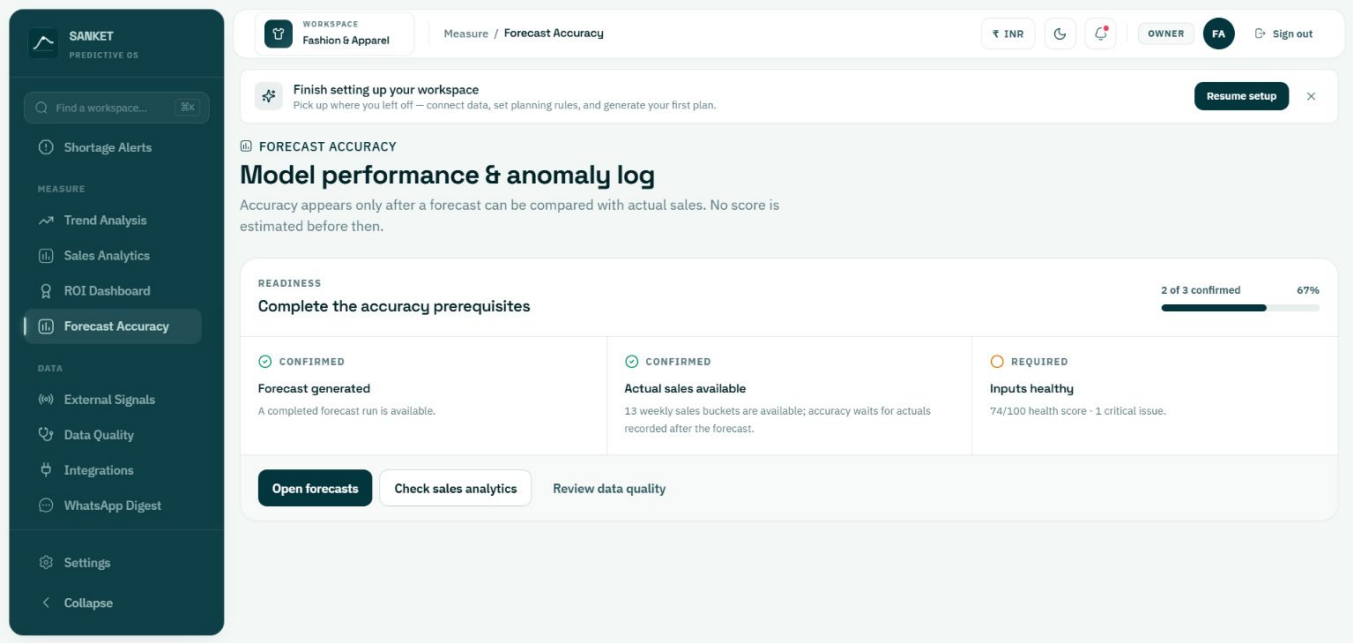
3. Inventory: totals reconcile, but pending cover can look safe.



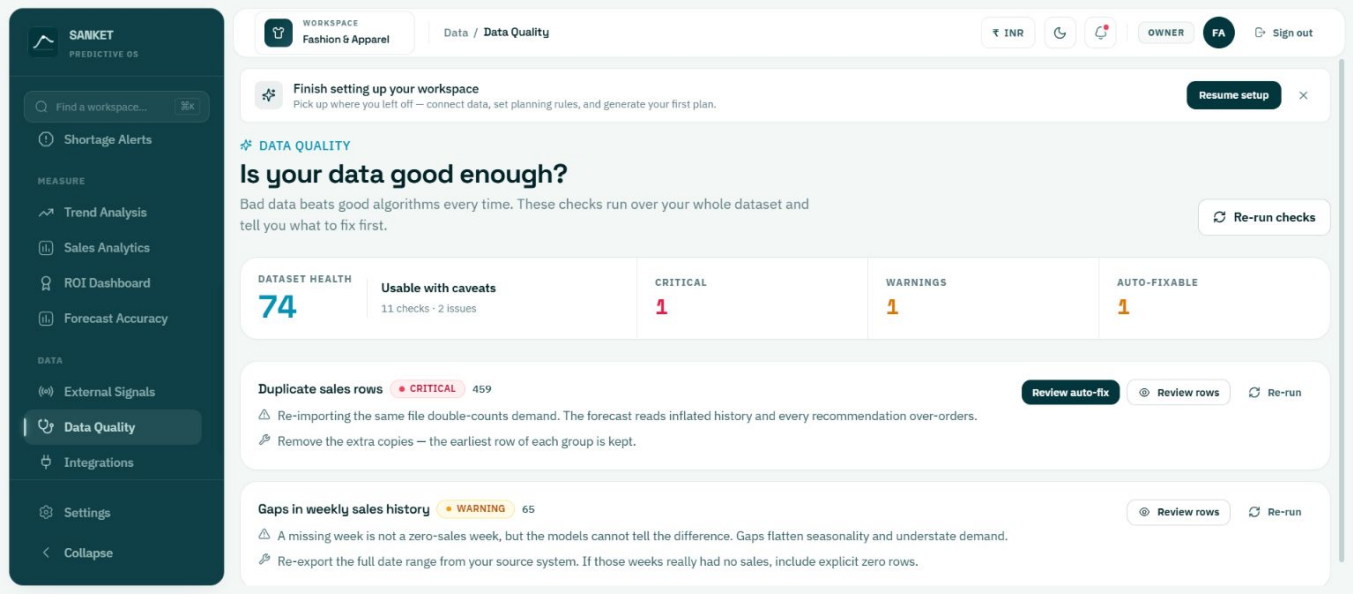
4. Scenario: demand preset finishes and changes the comparison.



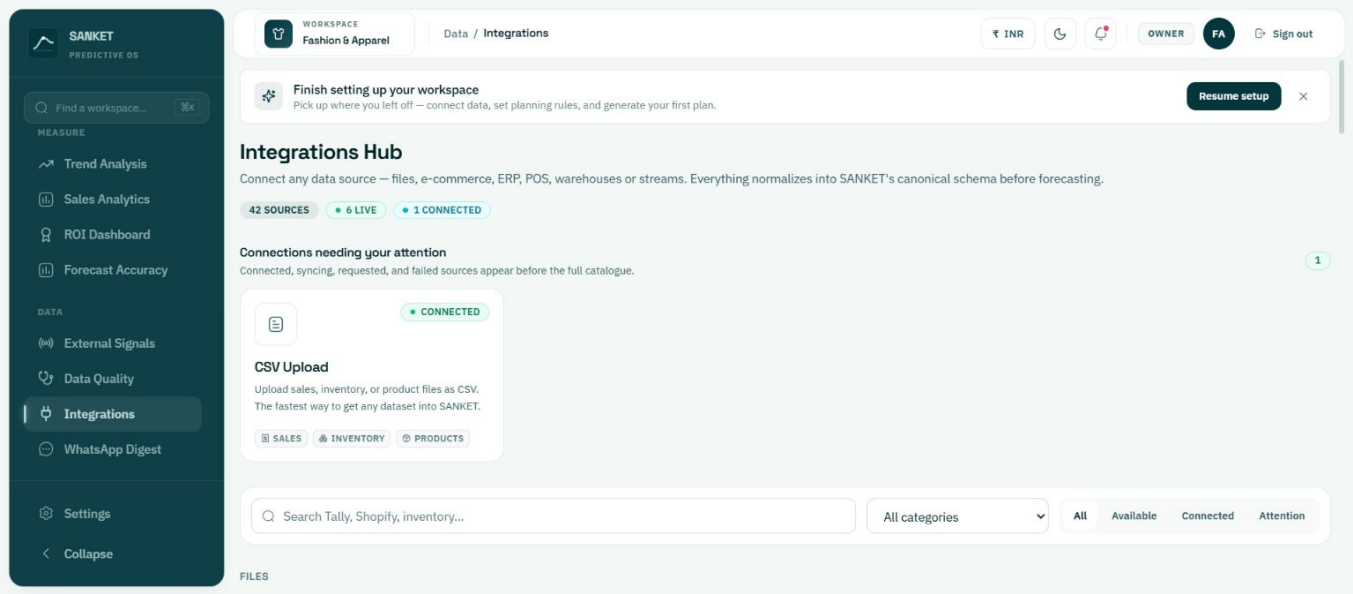
5. Accuracy: prerequisites shown, no invented measured score.



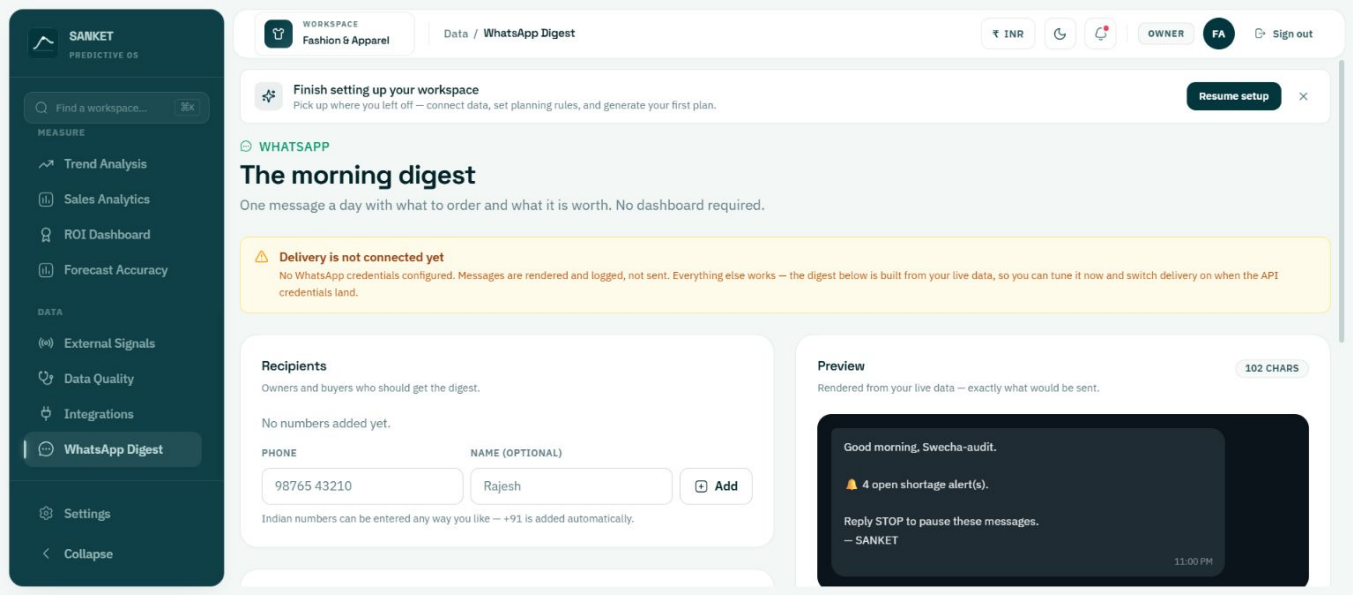
6. Data quality correctly identifies 459 duplicate copies.



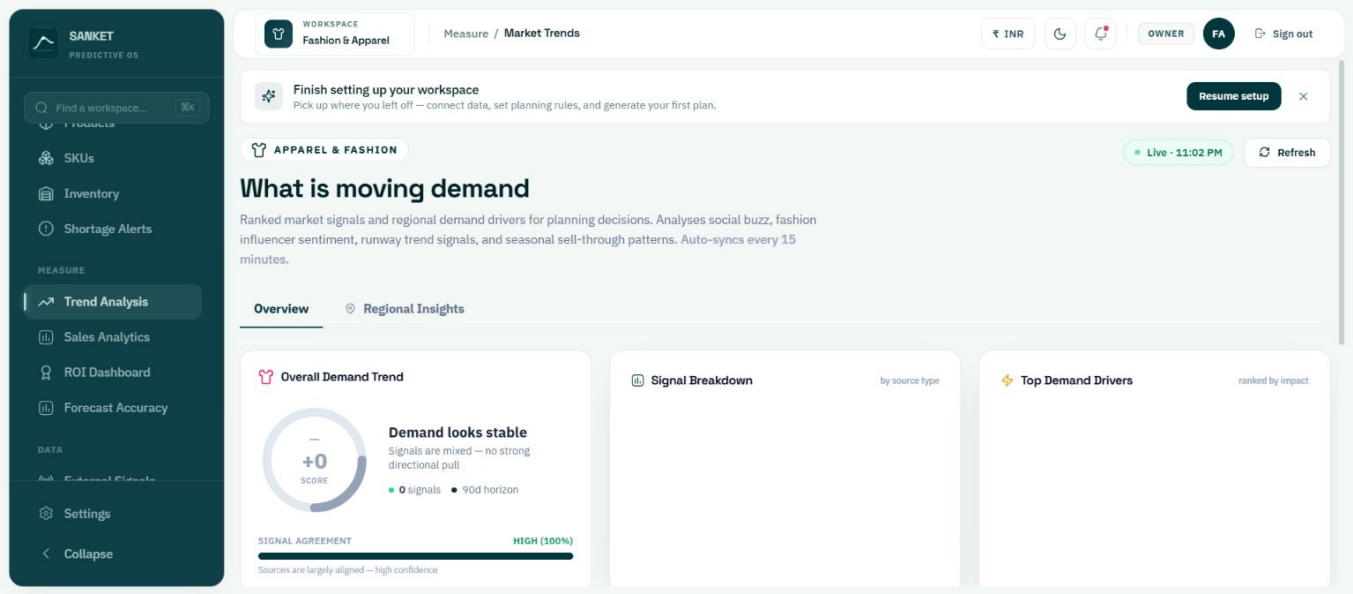
7. Integrations distinguishes live sources from future connectors.



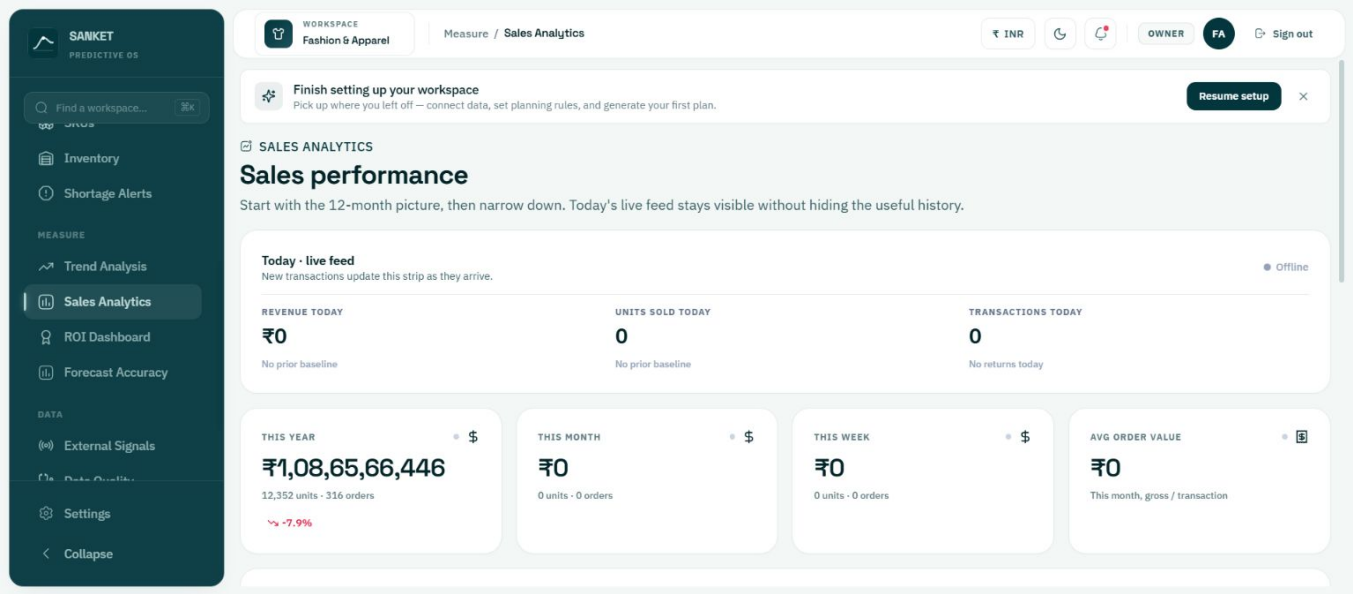
8. WhatsApp correctly discloses that delivery is not configured.



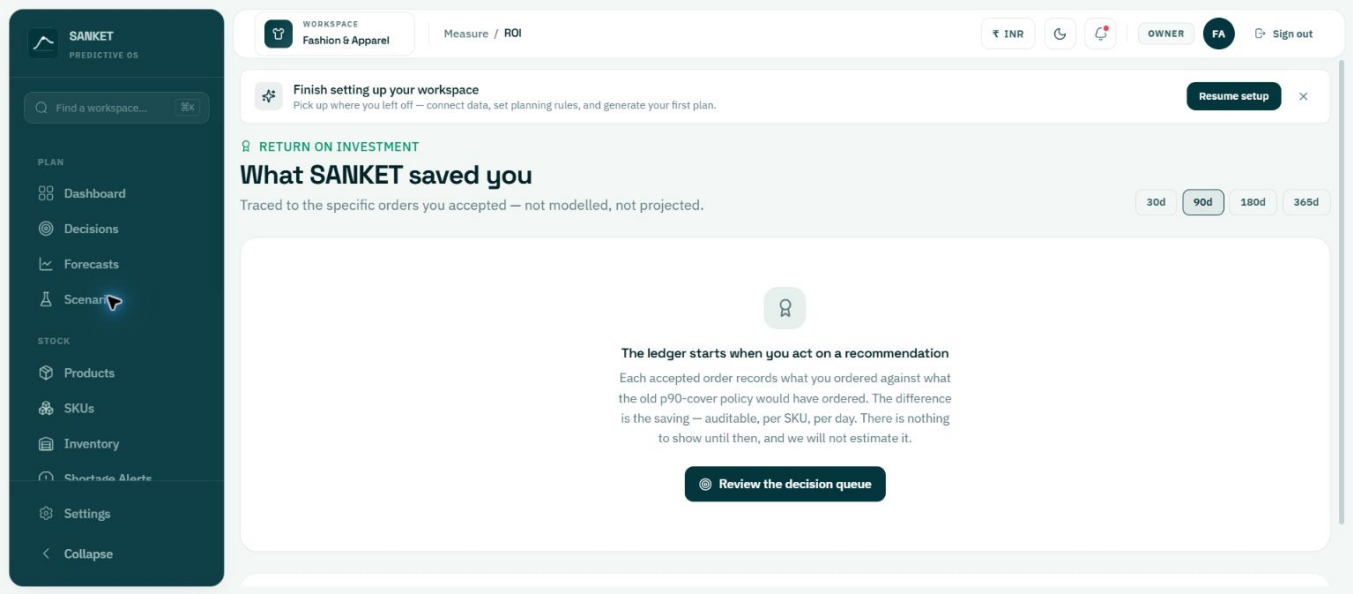
9. Zero signals incorrectly show 100% agreement.



10. INR sales display is inflated 83x by currency conversion.



11. ROI claims observed savings; approval-linked values are estimates.



10. Evidence standard

The verdict distinguishes execution, correctness and applicability. A 200 response proves only that a request completed; it does not prove the result is economically correct. Tests passing do not cancel reproduced defects. Synthetic holdout performance establishes that a real model runs on controlled inputs; it cannot establish Swecha's future accuracy. The report recommends a bounded pilot precisely because the implementation has useful working parts and material gaps at the same time.